

# **STUDENT MANAGEMENT SYSTEM**

## **Mini Project Report**

*Submitted for*

**BCA Final Year - Mini Project Assignment**

Language / Tech Used: C++ (Linked List based CRUD application)

Submission Date: Monday, 3 August 2026

**Pratibha Singh**

Roll No: 24116001192

Section: C

Axis Institute of Higher Education

## 1. Introduction

The Student Management System is a menu-driven console application developed in C++ to manage student records efficiently. It replaces manual, paper-based record keeping with a simple digital system capable of performing the core CRUD operations: Create (Add), Read (Search/Display), Update, and Delete. The application is built around a singly linked list, a fundamental linear data structure, which stores each student's details as a node and allows records to be inserted, searched, and removed dynamically without a fixed size limitation.

## 2. Objectives

- To design a menu-driven application that performs Add, Search, Update, and Delete operations on student records.
- To implement and apply a linked list data structure for dynamic storage of records.
- To provide persistent storage so that data is retained between program runs using file handling.
- To strengthen understanding of object-oriented programming, pointers, and file I/O in C++.

## 3. Tools and Technologies Used

| Component         | Details                                                 |
|-------------------|---------------------------------------------------------|
| Language          | C++ (C++17 standard)                                    |
| Data Structure    | Singly Linked List (sorted by Roll No)                  |
| Compiler          | g++ (GNU C++ Compiler)                                  |
| Storage           | Text file (students.dat) for persistence                |
| IDE / Environment | Any C++ IDE / Terminal (Code::Blocks, VS Code, Dev-C++) |

## 4. Data Structure Used: Singly Linked List

Each student record is stored as a node containing the fields Roll No, Name, Course, Semester, CGPA, Contact, and a pointer to the next node. New students are inserted in ascending order of Roll No, keeping the list sorted at all times. A linked list was chosen instead of a static array because it grows and shrinks dynamically at runtime, and insertion/deletion do not require shifting of other elements, unlike an array-based implementation.

## 5. System Modules (CRUD Operations)

**Add Student:** Inserts a new node into the linked list at the correct sorted position after checking that the Roll No does not already exist.

**Display All Students:** Traverses the linked list from head to tail and prints every record in a tabular format.

**Search Student:** Traverses the list comparing each node's Roll No until a match is found or the list ends.

**Update Student:** Locates the node by Roll No and modifies its Name, Course, Semester, CGPA, and Contact fields.

**Delete Student:** Locates the node, re-links the previous node to the next node, and frees the deleted node's memory.

**File Persistence:** Data is loaded from students.dat when the program starts and saved back after every Add, Update, or Delete operation, so records are not lost when the program closes.

## 6. Program Flow

- Program starts and loads any existing records from students.dat into the linked list.
- The main menu is displayed in a loop, offering 6 choices (Add, Display, Search, Update, Delete, Exit).

- Based on the user's choice, the corresponding function is called to operate on the linked list.
- After any operation that changes data, the list is immediately saved back to the file.
- The loop continues until the user selects Exit (option 6), at which point the program saves and terminates.

## 7. Sample Output / Screenshots

The following screenshots demonstrate the working of each menu operation during a sample run of the program.

```

student@bca-lab: ~/StudentManagementSystem

=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student (by Roll No)
4. Update Student
5. Delete Student
6. Exit
=====
Enter your choice (1-6): 1

--- Add New Student ---
Enter Roll No: 101
Enter Name: Aman Sharma
Enter Course: BCA
Enter Semester: 5
Enter CGPA: 8.5
Enter Contact No: 9876543210

Student added successfully!

```

*Fig 1: Adding a new student record*

```

student@bca-lab: ~/StudentManagementSystem

=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student (by Roll No)
4. Update Student
5. Delete Student
6. Exit
=====
Enter your choice (1-6): 2

=====
Roll    Name        Course    Sem  CGPA    Contact
-----
101     Aman Sharma  BCA       5     8.50    9876543210
102     Priya Singh  BCA       5     9.10    9123456780
103     Rahul Verma  BSc CS    3     7.80    9988776655
=====
Total Students: 3

```

*Fig 2: Displaying all student records*

```
student@bca-lab: ~/StudentManagementSystem

=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student (by Roll No)
4. Update Student
5. Delete Student
6. Exit
=====
Enter your choice (1-6): 3

--- Search Student ---
Enter Roll No to search: 102

--- Student Found ---
Roll No   : 102
Name      : Priya Singh
Course    : BCA
Semester  : 5
CGPA      : 9.10
Contact   : 9123456780
```

*Fig 3: Searching a student by Roll No*

```
student@bca-lab: ~/StudentManagementSystem

=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student (by Roll No)
4. Update Student
5. Delete Student
6. Exit
=====
Enter your choice (1-6): 4

--- Update Student ---
Enter Roll No to update: 102
Current details -> Name: Priya Singh, Course: BCA, Semester: 5, CGPA: 9.10, Contact: 9123456780
Enter new details:
Enter Name: Priya Singh
Enter Course: BCA
Enter Semester: 6
Enter CGPA: 9.4
Enter Contact No: 9123456780

Student record updated successfully!
```

*Fig 4: Updating an existing student record*

```
student@bca-lab: ~/StudentManagementSystem

=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student (by Roll No)
4. Update Student
5. Delete Student
6. Exit
=====
Enter your choice (1-6): 5

--- Delete Student ---
Enter Roll No to delete: 101
Are you sure you want to delete Roll No 101? (y/n): y

Student deleted successfully!

=====
STUDENT MANAGEMENT SYSTEM
=====
1. Add Student
2. Display All Students
3. Search Student (by Roll No)
4. Update Student
5. Delete Student
6. Exit
=====
Enter your choice (1-6): 6

Data saved. Exiting program. Goodbye!
```

*Fig 5: Deleting a student record and exiting*

## 8. Conclusion

The Student Management System successfully demonstrates the practical application of a linked list data structure to manage dynamic records through a menu-driven C++ console program. All the required CRUD operations were implemented and tested, and file handling was added so that data persists across program runs. The project reinforced core programming concepts such as structures, pointers, dynamic memory management, and file I/O in C++.

## 9. Future Scope

- Add a graphical user interface (GUI) using Qt or similar frameworks.
- Integrate with a proper database (SQLite/MySQL) instead of a flat text file.
- Add authentication (admin login) before allowing modifications.
- Add sorting/filtering by course, semester, or CGPA, and export reports to PDF/CSV.

## 10. Appendix: Source Code

The complete C++ source code (StudentManagementSystem.cpp) is listed below.

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <iomanip>
#include <cstdlib>
#include <limits>

using namespace std;

const string DATA_FILE = "students.dat";

struct Student {
    int rollNo;
    string name;
    string course;
    int semester;
    float cgpa;
    string contact;
    Student* next;
};

class StudentDatabase {
private:
    Student* head;
    int count;

public:
    StudentDatabase() : head(nullptr), count(0) {}

    ~StudentDatabase() {
        clearAll();
    }

    bool isEmpty() const { return head == nullptr; }
    int getCount() const { return count; }

    bool addStudent(int rollNo, const string& name, const string& course,
                    int semester, float cgpa, const string& contact) {
        if (findStudent(rollNo) != nullptr) {
            return false;
        }

        Student* newNode = new Student{rollNo, name, course, semester, cgpa, contact, nullptr};

        if (head == nullptr || head->rollNo > rollNo) {
            newNode->next = head;
            head = newNode;
        } else {
            Student* temp = head;
            while (temp->next != nullptr && temp->next->rollNo < rollNo) {
```

```

        temp = temp->next;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}
count++;
return true;
}

Student* findStudent(int rollNo) const {
    Student* temp = head;
    while (temp != nullptr) {
        if (temp->rollNo == rollNo) return temp;
        temp = temp->next;
    }
    return nullptr;
}

bool updateStudent(int rollNo, const string& name, const string& course,
                  int semester, float cgpa, const string& contact) {
    Student* s = findStudent(rollNo);
    if (s == nullptr) return false;
    s->name = name;
    s->course = course;
    s->semester = semester;
    s->cgpa = cgpa;
    s->contact = contact;
    return true;
}

bool deleteStudent(int rollNo) {
    Student* temp = head;
    Student* prev = nullptr;

    while (temp != nullptr && temp->rollNo != rollNo) {
        prev = temp;
        temp = temp->next;
    }

    if (temp == nullptr) return false;

    if (prev == nullptr) {
        head = temp->next;
    } else {
        prev->next = temp->next;
    }

    delete temp;
    count--;
    return true;
}

void displayAll() const {
    if (isEmpty()) {

```

```

        cout << "\nNo student records found.\n";
        return;
    }

    cout << "\n" << string(90, '=') << "\n";
    cout << left << setw(8) << "Roll"
        << setw(20) << "Name"
        << setw(15) << "Course"
        << setw(6) << "Sem"
        << setw(8) << "CGPA"
        << setw(15) << "Contact" << "\n";
    cout << string(90, '-') << "\n";

    Student* temp = head;
    while (temp != nullptr) {
        cout << left << setw(8) << temp->rollNo
            << setw(20) << temp->name
            << setw(15) << temp->course
            << setw(6) << temp->semester
            << setw(8) << fixed << setprecision(2) << temp->cgpa
            << setw(15) << temp->contact << "\n";
        temp = temp->next;
    }
    cout << string(90, '=') << "\n";
    cout << "Total Students: " << count << "\n";
}

void clearAll() {
    Student* temp = head;
    while (temp != nullptr) {
        Student* next = temp->next;
        delete temp;
        temp = next;
    }
    head = nullptr;
    count = 0;
}

void saveToFile() const {
    ofstream fout(DATA_FILE);
    if (!fout) {
        cout << "Error: could not open file for saving.\n";
        return;
    }
    Student* temp = head;
    while (temp != nullptr) {
        fout << temp->rollNo << "|" << temp->name << "|" << temp->course << "|"
            << temp->semester << "|" << temp->cgpa << "|" << temp->contact << "\n";
        temp = temp->next;
    }
    fout.close();
}

void loadFromFile() {

```



```

    ifstream fin(DATA_FILE);
    if (!fin) return;

    clearAll();
    string line;
    while (getline(fin, line)) {
        if (line.empty()) continue;
        stringstream ss(line);
        string token;
        int rollNo, semester;
        string name, course, contact;
        float cgpa;

        getline(ss, token, '|'); rollNo = stoi(token);
        getline(ss, name, '|');
        getline(ss, course, '|');
        getline(ss, token, '|'); semester = stoi(token);
        getline(ss, token, '|'); cgpa = stof(token);
        getline(ss, contact, '|');

        addStudent(rollNo, name, course, semester, cgpa, contact);
    }
    fin.close();
};

int readInt(const string& prompt) {
    int value;
    while (true) {
        cout << prompt;
        if (cin >> value) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
        cout << "Invalid input. Please enter a number.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

float readFloat(const string& prompt) {
    float value;
    while (true) {
        cout << prompt;
        if (cin >> value) {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            return value;
        }
        cout << "Invalid input. Please enter a valid number.\n";
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
}

```

```

string readLine(const string& prompt) {
    string value;
    cout << prompt;
    getline(cin, value);
    return value;
}

void addStudentMenu(StudentDatabase& db) {
    cout << "\n--- Add New Student ---\n";
    int rollNo = readInt("Enter Roll No: ");
    string name = readLine("Enter Name: ");
    string course = readLine("Enter Course: ");
    int semester = readInt("Enter Semester: ");
    float cgpa = readFloat("Enter CGPA: ");
    string contact = readLine("Enter Contact No: ");

    if (db.addStudent(rollNo, name, course, semester, cgpa, contact)) {
        db.saveToFile();
        cout << "\nStudent added successfully!\n";
    } else {
        cout << "\nError: Roll No " << rollNo << " already exists.\n";
    }
}

void searchStudentMenu(StudentDatabase& db) {
    cout << "\n--- Search Student ---\n";
    int rollNo = readInt("Enter Roll No to search: ");
    Student* s = db.findStudent(rollNo);

    if (s == nullptr) {
        cout << "\nStudent with Roll No " << rollNo << " not found.\n";
        return;
    }

    cout << "\n--- Student Found ---\n";
    cout << "Roll No   : " << s->rollNo << "\n";
    cout << "Name       : " << s->name << "\n";
    cout << "Course      : " << s->course << "\n";
    cout << "Semester    : " << s->semester << "\n";
    cout << "CGPA        : " << fixed << setprecision(2) << s->cgpa << "\n";
    cout << "Contact     : " << s->contact << "\n";
}

void updateStudentMenu(StudentDatabase& db) {
    cout << "\n--- Update Student ---\n";
    int rollNo = readInt("Enter Roll No to update: ");

    Student* s = db.findStudent(rollNo);
    if (s == nullptr) {
        cout << "\nStudent with Roll No " << rollNo << " not found.\n";
        return;
    }

    cout << "Current details -> Name: " << s->name << ", Course: " << s->course

```

```

        << " , Semester: " << s->semester << " , CGPA: " << s->cgpa
        << " , Contact: " << s->contact << "\n";
    cout << "Enter new details:\n";

    string name = readLine("Enter Name: ");
    string course = readLine("Enter Course: ");
    int semester = readInt("Enter Semester: ");
    float cgpa = readFloat("Enter CGPA: ");
    string contact = readLine("Enter Contact No: ");

    db.updateStudent(rollNo, name, course, semester, cgpa, contact);
    db.saveToFile();
    cout << "\nStudent record updated successfully!\n";
}

void deleteStudentMenu(StudentDatabase& db) {
    cout << "\n--- Delete Student ---\n";
    int rollNo = readInt("Enter Roll No to delete: ");

    char confirm;
    cout << "Are you sure you want to delete Roll No " << rollNo << "? (y/n): ";
    cin >> confirm;
    cin.ignore(numeric_limits<streamsize>::max(), '\n');

    if (confirm != 'y' && confirm != 'Y') {
        cout << "Delete cancelled.\n";
        return;
    }

    if (db.deleteStudent(rollNo)) {
        db.saveToFile();
        cout << "\nStudent deleted successfully!\n";
    } else {
        cout << "\nStudent with Roll No " << rollNo << " not found.\n";
    }
}

void printMenu() {
    cout << "\n===== \n";
    cout << "        STUDENT MANAGEMENT SYSTEM\n";
    cout << "===== \n";
    cout << "1. Add Student\n";
    cout << "2. Display All Students\n";
    cout << "3. Search Student (by Roll No)\n";
    cout << "4. Update Student\n";
    cout << "5. Delete Student\n";
    cout << "6. Exit\n";
    cout << "===== \n";
}

int main() {
    StudentDatabase db;
    db.loadFromFile();

```

```
int choice;
do {
    printMenu();
    choice = readInt("Enter your choice (1-6): ");

    switch (choice) {
        case 1: addStudentMenu(db); break;
        case 2: db.displayAll(); break;
        case 3: searchStudentMenu(db); break;
        case 4: updateStudentMenu(db); break;
        case 5: deleteStudentMenu(db); break;
        case 6:
            db.saveToFile();
            cout << "\nData saved. Exiting program. Goodbye!\n";
            break;
        default:
            cout << "\nInvalid choice! Please enter a number between 1 and 6.\n";
    }
} while (choice != 6);

return 0;
}
```